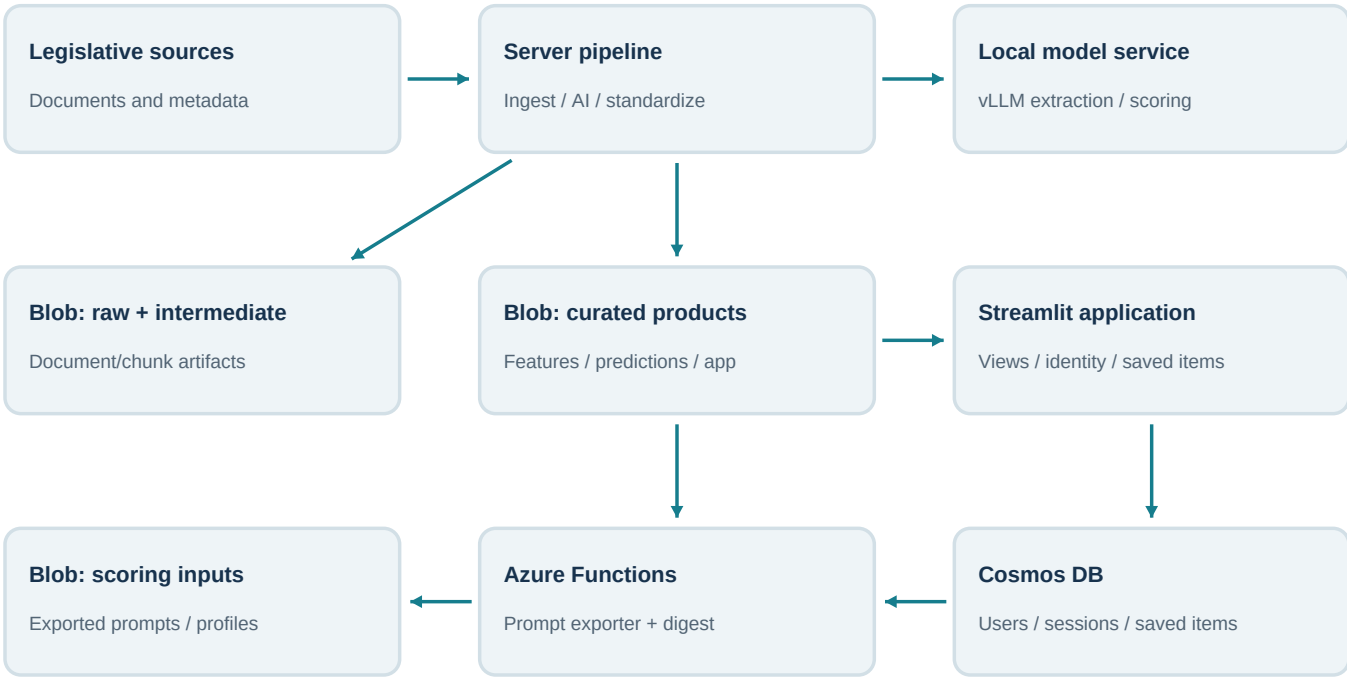
LEGISLATIVE INTELLIGENCE

System architecture and operator handoff

Server AI processing, cloud data products, a Streamlit web application and scheduled Azure Functions.



Additional paths: server consumes exported inputs; app/digest use email and Azure OpenAI.

Scope and evidence

This public-safe overview accompanies the original-source-derived repository. It describes inspected implementation and owner-supplied Azure metadata. It excludes credentials, user records, internal deployment locations, raw datasets and experimental work.

The web-app source matches the supplied successful release revision. Exact server schedules and the active Function source/package identity still require verification. This document does not claim new end-to-end runtime tests.

01 How information moves

The server processes legislative documents through ingestion, chunking, model extraction, synthesis and standardization. Curated results feed prediction, personalization and app-ready snapshots. The application reads those products rather than reproducing the entire document-processing workload.

Boundary	Implementation starting point	Maintainer question
Core document stages	components/pipeline/stages/	Which schema/model/config version produced this result?
Model and storage adapters	services/vllm_service.py; services/azure_blob_service.py	Which identities and artifact versions are required?
Prediction / change history	run_daily.py; stage2_transformers.py; build_diffs.py; build_probability_trail.py	Which source changes require rescore or republish?
App-ready publication	build_app_snapshot.py	Which app/digest catalog consumes this output?
Personalized scoring	score_impact_prompts.py; investigate_flagged_bills_v2.py	Are user, prompt and bill identifiers aligned?

Separate processing from deployment

The traced server wrapper gates model health, calls core orchestration, then invokes a separate personalization wrapper. That source relationship is established; the actual live scheduler entry is not. Alternative entry points and retraining scripts are retained in preservation storage, not selected for this release.

Model weights, fitted artifacts, datasets and completed outputs stay outside Git. A source checkout is not a complete recovery of the production environment. Record their approved storage, revision, restore procedure and owner before transferring maintenance.

02 Application, profiles and cloud work

The Streamlit application does more than display results. It implements navigation, magic-link/session logic, Cosmos persistence, saved items, team collaboration, filtered data loading, AI briefings and email integration.

Component	Where to start	Key responsibility
Web entry and pages	components/webapp/app.py; views/	Home, Tracker, Trends, Explorer, Active Bills, Validation and Roadmap
Identity / persistence	user_auth.py; auth_utils.py; user_profile.py; team_service.py	Sessions, user records, saved items and teams
Data-product loading	data_catalog.py; impact_loader.py; predictions_loader.py	Configured product locations, caching and consumer-side filtering
Prompt export Function	components/functions/prompt_exporter/	Read Cosmos prompt/profile inputs and publish the server scoring input
Digest Function	components/functions/digest_timer/	Read user and legislative data, generate/send email and update state

User-specific data needs its own boundary

Exported prompts, profiles, per-user results and diagnostic trails remain sensitive even when stored beside legislative data. Source-level dataframe filtering is not proof of complete authorization. Validate authenticated callers, storage permissions, caching boundaries and deletion/retention behavior separately.

Function helpers and catalogs are intentionally packaged separately from web helpers. Updating one copy does not update the other deployment. The saved local deployment ZIP differs from current digest/email source, so live package identity remains an open evidence item.

03 Operating and updating the system

Component	Observed release path	Before changing it
Web app	Local Git push to Azure; remote Oryx Python build	Use the web component as package root. Review dependencies/settings and test an isolated release.
Functions	Azure CLI ZIP deployment with remote Oryx build	Match the active package to source, confirm both timer settings, preserve state and a rollback package.
Server	Inspected model-health/orchestration wrappers	Confirm actual scheduler/timezone, active wrapper and replay behavior.

Safe checks first

python tools/verify_release.py

The release checker validates source syntax, source-manifest hashes, local documentation links and excluded/credential patterns without importing production modules. It does not start the app, call models, access Azure or send email.

Common failure paths

Stale general results: inspect stage completion, published artifact integrity and app caches. Stale personalized results: inspect exporter freshness, scorer/investigator logs and user-specific result publication. Repeating digest content: inspect state writes and freeze/test settings; state-freeze does not mean email dry-run.

Changing code is not changing every deployment

GitHub is the source/document home. A push here does not deploy the server, app or Functions. The monorepo root differs from the original web deployment root; never push it blindly to the Azure remote. A code rollback does not undo Cosmos changes or Blob overwrites.

04 Handoff evidence and recovery

An incoming collaborator should be able to locate every component, explain a synthetic record end to end, make an isolated change, diagnose a failed component handoff and recover the required state. Documentation is a starting point; demonstrate those tasks with a second authorized operator.

Established	Still required
Verified original server archive/file hashes and preserved local web source	Full restoration of source, approved configuration, model artifacts and required datasets
App release revision, runtime/startup evidence and named enabled Functions	Recent successful invocations, exact active Function source and server schedule
Cosmos container metadata and reported backup settings	Authorization/partition behavior, deliberate retention, tested recovery
Blob container inventory and reported soft-delete settings	Output existence/freshness (metadata access was denied), versioned recovery and access review
Scrubbed source subset with inclusion/exclusion provenance	Resolve use of extra Cosmos results storage, additional Blob container and provisioned Search resource

Canonical repository documents

README.md: orientation. docs/SYSTEM_GUIDE.md: architecture, components and contracts. docs/OPERATIONS.md: configuration, update/recovery workflow and unresolved evidence. docs/SOURCE_MANIFEST.json: provenance and scrubbing. docs/knowledge-index.json: navigation for future question-answering tools.

A future Teams agent can cite these sources and explicitly distinguish inspected code, observed settings and tested behavior. No Teams agent or automated operational capability is deployed by this release.

Public portfolio claims should describe implemented engineering and observed evidence. Do not invent adoption, accuracy, throughput, savings or availability metrics. Exact resource identities and owner/access procedures remain in the restricted operator register.